

GitSkills: A Dataset of Agent Skills on GitHub

Giuseppe Destefanis*, Daniel Graziotin†, Matteo Vaccargiu‡, Marco Ortu‡

*University College London, United Kingdom g.destefanis@ucl.ac.uk

†University of Hohenheim, Stuttgart, Germany {graziotin, matteo.vaccargiu}@uni-hohenheim.de

‡University of Cagliari, Italy marco.ortu@unica.it

Abstract—An agent skill is a folder containing a `SKILL.md` file with instructions for a language-model agent, optionally accompanied by scripts and reference files. The agent loads the skill when it judges that a task matches the skill description. Anthropic introduced the format in October 2025 as an open specification. Nine months later, public GitHub repositories hold millions of skill files. Skills are unlike the artifacts that software engineering researchers usually mine: they are written mainly in natural language, a model selects them probabilistically at run time, and no compiler or type checker verifies the selection. Skills also have no central registry or package manager; developers reuse them by copying folders between repositories. How developers write, reuse, and maintain skills is therefore an empirical question, and no existing dataset records this population. We present GitSkills, a dataset of 3,797,117 `SKILL.md` files collected from 282,200 public repositories in July 2026. The dataset retains every file occurrence with its repository, path, and content hash. We group identical files into 1,877,981 distinct contents and enrich one representative per group with the full text, parsed front matter, folder contents, repository metadata, and, for a subset, the commit history of the file. A single self-contained SQLite file supports research on the adoption, reuse, structure, authorship, maintenance, and security of agent skills.

Index Terms—mining software repositories, LLM agents, agent skills, datasets, software artifacts

I. HIGH-LEVEL OVERVIEW

An agent skill describes a workflow, convention, or procedure that a language-model agent should follow for a class of tasks. A skill consists of a folder containing a `SKILL.md` file. The file has YAML front matter with a name and description, followed by a Markdown body with the instructions (Figure 2 in the Appendix). The folder may also contain scripts and reference documents. Anthropic introduced the format in October 2025 and published its directory structure, front-matter constraints, and staged loading model as an open specification [1], [2]. Any agent tool can support the format. Claude Code is the reference implementation [3]; where a tool-specific convention matters, such as the `.claude/skills/` skill directory, we follow Claude Code’s for simplicity.

The model selects skills probabilistically: at run time it compares the task with the skill description and decides whether to load the skill, a step that no compiler or type checker verifies. A vague description may prevent selection, while unclear instructions may lead to incomplete or incorrect execution without an explicit error. These problems are difficult to detect with conventional software analysis. How developers write and maintain skills is therefore an empirical question, but no dataset currently supports a large-scale study of this artifact.

GitSkills records 3,797,117 `SKILL.md` files from 282,200 public GitHub repositories owned by 195,841 accounts, collected in July 2026. We group the files by content hash into 1,877,981 distinct contents and enrich one representative file per group with its full text, parsed front matter, folder contents, and repository metadata; commit history, with first- and last-commit author accounts, covers skills in standard locations (recognized skill directories such as `.claude/skills/`) and a size-stratified sample of the rest. Every occurrence is retained with its repository and path, so researchers can study both unique contents and their copies.

The dataset covers the early adoption of the format, while conventions and tooling are still developing, so researchers can observe how a new software artifact spreads, how common practices emerge, and whether the format develops into shared infrastructure across agent tools.

We collected the dataset with a read-only pipeline against the GitHub code-search and REST APIs. Because code search returns at most 1,000 results per query, discovery partitions the search space by file size until every range can be retrieved completely. Appendix A details the pipeline and its coverage limits; the dataset covers public repositories only and is a lower bound on the population.

II. INTERNAL STRUCTURE

Table I summarizes the dataset tables. The dataset is stored in SQLite format.

When several repositories contain the same `SKILL.md` file, the dataset stores the text once and links all copies to it through the content hash. Identical hashes mean that the files contain exactly the same bytes.

We collect folder contents and commit history for only one copy of each distinct skill. This information applies only to that specific repository. Other copies may have different scripts, reference files, or commit histories.

III. POTENTIAL RESEARCH QUESTIONS

Agent skills influence how coding agents operate within software projects, so their content, structure, reuse, and maintenance are empirical questions, which researchers can now study at the level of the population.

1) Adoption and linguistic evolution.

- How quickly does the format spread, and which projects adopt it first, in terms of programming language, popularity, age, and activity?

Table I
OVERVIEW OF THE GITSKILLS DATASET.

	Table	# Records	Content
Core	artifacts	3,797,117	One row per discovered file: repository, path, exact basename, location class, content hash, and representative flag; for representatives, also the full text, parsed front matter, and body size.
	repos	282,200	Repository metadata: owner, star count, primary language, fork status, creation date, and last-push date.
Composition	artifact_siblings	7,264,865	Files stored alongside a representative skill: path, entry type (file or directory), size, and the text of files under a size cap. Whether a skill bundles scripts or reference material is recorded per skill in artifacts (has_scripts, has_references).
History	(columns in artifacts)	458,548	First and last commit dates of the SKILL.md file, their author accounts (anonymized; user or bot), and commit count, for standard locations and a size-stratified sample of the rest.
Collection	mining_runs	7	Query, start and end timestamps, and search-result count for each collection run; four runs that crashed were closed retroactively with a zero count.

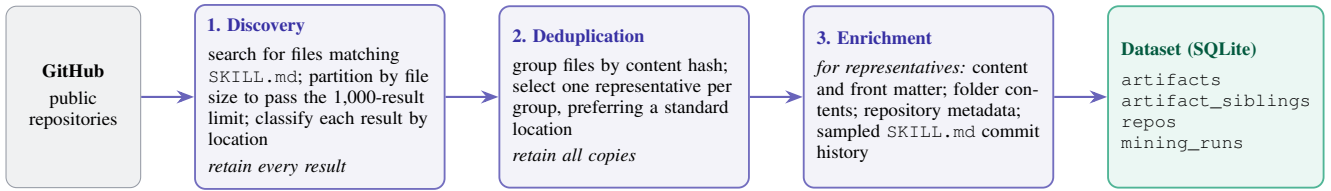


Figure 1. Collection pipeline. Discovery retains every filename match; deduplication selects one representative per distinct content while retaining all copies; enrichment applies to the representatives.

- b) What do developers codify in skills, and in which contexts do skills appear, from operational projects to catalogs, templates, and demonstrations? A taxonomy of skill purposes does not yet exist.
- c) Do the linguistic properties of newly written skills change across monthly cohorts, in structure and phrasing as well as in topic coverage and semantic diversity? Convergence toward formulaic templates would indicate an emerging genre; shrinking diversity may also reflect rising machine authorship.
- d) How many skills do agents themselves create or maintain, and in which natural languages are skills written? A skill is read by a multilingual model, so a developer may state a procedure more precisely in their own language than in English.

2) Development of a shared format.

- a) What proportion of skills use vendor-neutral rather than tool-specific locations, and how does this proportion change over time?
- b) Do skill texts address one named tool, or any agent that implements the specification?

3) Reuse without a package manager. Skills have no central registry; reuse happens by copying folders, and 50.5% of the collected files are byte-for-byte copies of another file in the dataset.

- a) How concentrated is reuse: a long tail of rarely copied contents, or a small set of widely copied templates?
- b) Through which mechanisms do skills move between repositories, such as direct addition, catalogs, or scaffolding tools?

- c) Do skill copies follow the genealogy patterns known from code clones, such as consistent and inconsistent propagation of changes?

4) Software metrics for natural-language instructions.

- a) Which established metrics, such as size, churn, age, clone coverage, and readability, have meaningful equivalents for skills, and how do their distributions compare with those of source code?
- b) Can observable indicators of skill quality be defined and compared with proxies such as copy count and subsequent edits?
- c) Which properties of the description, the text the agent matches against when deciding whether to load the skill, are associated with reuse and maintenance?

5) Maintenance and trust. Skills can instruct agents to run commands, access external resources, and execute bundled scripts, and they are copied between repositories without formal review.

- a) How often do skills become outdated relative to the projects and tools they describe?
- b) Do modified copies of widely reused skills introduce command execution or network access absent from the original, the analog of a supply-chain attack in an ecosystem without a registry?
- c) How often do skills bundle executable files, and how widely are these skills copied?

IV. HOW TO ACCESS

The full dataset, as a single self-contained SQLite file, is archived on Zenodo at this link¹. A Parquet mirror, partitioned by table, is available on Hugging Face at this link².

A sample of the dataset is available on GitHub at this link³.

We replaced commit author accounts with keyed one-way codes, identical for the same account throughout, so authorship can be traced without identifying anyone. Bot accounts keep their login. We redacted email addresses and personal names in commit messages and kept AI assistant names in Co-authored-by trailers.

Acknowledgement. Matteo Vaccargiu has been supported by the Hector Stiftung.

REFERENCES

- [1] Agent Skills, “Agent Skills Specification (open standard, Apache-2.0),” <https://agentskills.io>.
- [2] Anthropic, “Agent Skills,” Claude API documentation, <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>.
- [3] Anthropic, “Skills in Claude Code,” Claude Code documentation, <https://code.claude.com/docs/en/skills>.
- [4] G. Destefanis, “Authoring Agent Skills: A Software-Engineering Approach,” *arXiv preprint arXiv:2607.25032*, 2026.

APPENDIX A

DATASET CONSTRUCTION

Figure 1 shows the collection pipeline. Collection is read-only; requests go to the GitHub REST and GraphQL APIs, the code-search API, and the raw-content CDN. Each stage checkpoints its progress in the database and resumes after interruptions.

Discovery. Agent tools identify skills by filename, so the exact basename is a direct marker for code search. The code-search API returns at most 1,000 results per query, and its `total_count` estimate proved unreliable: it reported roughly 349,000 matches for the filename query, against over 3.8M files ultimately retrieved. We therefore partitioned the search space by file size, splitting any range with more than 1,000 results until every range could be retrieved completely.

Deduplication. We group files by content hash and select one representative per group for enrichment, preferring a file in the `.claude/skills/` directory, with a deterministic rule to break ties. We do not assume that the representative is the original source. All copies remain in the dataset with their repository, path, and location class, so the spread of each content can be measured.

Enrichment. For each representative, the tool downloads the `SKILL.md` file, parses its front matter, and records the bundled scripts and reference documents in the skill folder, downloading the text of bundled files up to a size cap. It also collects repository metadata, including star counts, and retrieves the commit history of the `SKILL.md` file, with the author account of the first and last commit stored as an anonymized code, for skills in standard locations and a size-stratified

```
---
name: web-artifacts-builder
description: Suite of tools for creating elaborate, multi-
  component claude.ai HTML artifacts using modern frontend
  web technologies (React, Tailwind CSS, shadcn/ui). Use
  for complex artifacts requiring state management, routing
  , or shadcn/ui components - not for simple single-file
  HTML/JSX artifacts.
license: Complete terms in LICENSE.txt
---

# Web Artifacts Builder

To build powerful frontend claude.ai artifacts, follow
these steps:
1. Initialize the frontend repo using 'scripts/init-
  artifact.sh'
2. Develop your artifact by editing the generated code
3. Bundle all code into a single HTML file using 'scripts/
  bundle-artifact.sh'
4. Display artifact to user
5. (Optional) Test the artifact

[...]
```

Figure 2. Opening of the `SKILL.md` of Anthropic’s `web-artifacts-builder` skill, reproduced verbatim and truncated. Source: <https://github.com/anthropics/skills>. The YAML front matter carries the name and the description that the agent matches against the task when deciding whether to load the skill; the Markdown body holds the instructions, here referencing bundled executable scripts.

sample of the others; the file’s own history dates the skill’s addition. Folder listings are missing for 1,544 representatives and commit history for 1,166 of those in standard locations. For 42 representatives the file changed on GitHub between discovery and fetch and the original could not be recovered; `content_sha_ok` records this.

Anonymization. We masked email addresses in commit messages, including GitHub noreply addresses, with a fixed marker, and a scan of the released file confirmed that none remain; the first and last commit messages are included in this redacted form. We replaced commit author accounts and personal names in trailer lines with codes from a keyed one-way function; the codes cannot be reversed and stay stable across the dataset.

The search matches the basename case-insensitively and also returns case variants such as `skill.md` (54,289 files, 1.4%), including lowercase files predating the format. We retained them: each record carries the exact basename, location class, front-matter validity, and date of the first recorded commit, so researchers can define and compare stricter inclusion criteria during analysis. The dataset covers public repositories only, and GitHub code search indexes only default branches, files under 384 KB, recently active repositories with fewer than 500,000 files, and forks only when they have more stars than the parent repository. The dataset is therefore a lower bound on the full population.

¹<https://doi.org/10.5281/zenodo.21875637>

²<https://huggingface.co/datasets/mvaccargiu/gitskills>

³<https://github.com/giuseppedestefanis/gitskills-sample>